

Исследование команд Git в терминале

Отчёт по учебной практике. Задание 2.3. Вариант 4.

Качков Кирилл Евгеньевич, группа ИВТ-2.

Цель работы: исследовать основные команды Git в терминале Windows - создание репозитория, сохранение изменений, ветвление, слияние, отмену правок и обмен коммитами. Проверка выполнена в отдельной учебной папке с Git 2.54.0.windows.1.

В учебном проекте подготовлены README.md, plan.txt и .gitignore. Правило *.tmp исключает временные файлы из списка неотслеживаемых. Снимки ниже относятся к двум запускам одной демонстрации, поэтому хеши коммитов могут различаться.

Листинг 1 - создание репозитория и первый коммит

```
git --version
git init -b main
git status --short
git add README.md plan.txt .gitignore
git diff --cached --stat
git commit -m "Add initial practice plan"
```

Команда init создаёт репозиторий с веткой main. Статус ?? обозначает неотслеживаемые файлы. Команда add помещает их в индекс, diff --cached показывает подготовленные изменения, а commit сохраняет их в истории.

```
PRACTICE 2.3 / GIT CLI / STAGE 1
Repository initialization and first commit

$ git --version
git version 2.54.0.windows.1
$ git init -b main
Initialized empty Git repository in <DEMO>/student/.git/
$ git status --short
?? .gitignore
?? README.md
?? plan.txt
$ git add README.md plan.txt .gitignore

$ git diff --cached --stat
.gitignore | 1 +
README.md | 1 +
plan.txt | 1 +
3 files changed, 3 insertions(+)
$ git commit -m Add initial practice plan
[main (root-commit) 37dbb33] Add initial practice plan
3 files changed, 3 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 plan.txt
```

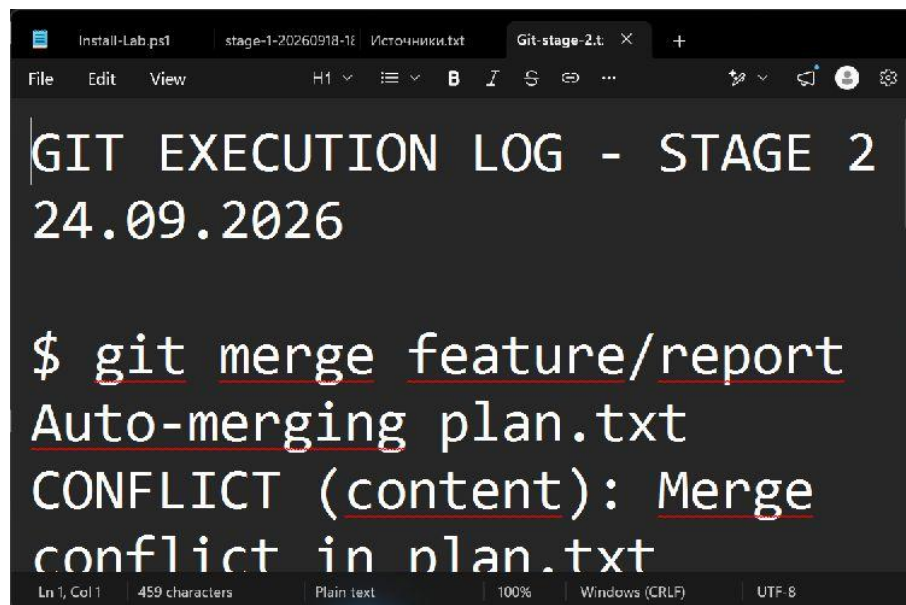
Создание репозитория и первый коммит

Для проверки ветвления в отдельной ветке изменено значение review в plan.txt. Затем другая версия этой строки сохранена в main. Изменение одной строки в двух ветках вызвало конфликт при слиянии.

Листинг 2 - работа с ветками и слияние

```
git switch -c feature/report
git diff -- plan.txt
git add plan.txt
git commit -m "Prepare report in feature branch"
git switch main
git add plan.txt
git commit -m "Update main review plan"
git merge feature/report
git status --short
```

Команда switch -c создаёт ветку и переключается в неё. Перед каждым коммитом файл plan.txt редактируется в соответствующей ветке. После merge Git сообщил о конфликте, а статус UU показал, что файл требует ручного объединения.

A screenshot of a terminal window with a dark background and light-colored text. The window title bar shows several tabs: 'Install-Lab.ps1', 'stage-1-20260918-1e', 'Источники.txt', and 'Git-stage-2.t'. The terminal content displays the output of a git merge command, showing a conflict in the file 'plan.txt'. The text is as follows:
GIT EXECUTION LOG - STAGE 2
24.09.2026

\$ git merge feature/report
Auto-merging plan.txt
CONFLICT (content): Merge
conflict in plan.txt
At the bottom of the terminal, status information is visible: 'Ln 1, Col 1', '459 characters', 'Plain text', '100%', 'Windows (CRLF)', and 'UTF-8'.

Сообщение о конфликте при слиянии веток

Для разрешения конфликта в файле оставлена строка review=checked-and-ready, а маркеры конфликта удалены. После этого результат добавлен в индекс и сохранён коммитом слияния.

Листинг 3 - сохранение результата разрешения конфликта

```
git add plan.txt
git commit -m "Resolve review plan conflict"
git status --short
```

После коммита конфликт устранин. Обе ветви истории сохранены, рабочая папка не содержит незакоммиченных изменений.

Следующий этап - проверка отмены изменений. В plan.txt временно изменена строка review. Команда diff показала правку, а restore вернула сохранённую версию файла.

Листинг 4 - отмена незакоммиченного изменения

```
git diff -- plan.txt
git restore plan.txt
git status --short
```

Повторный сокращённый статус оказался пустым: локальная правка отменена. Для проверки отмены уже сохранённого изменения в README.md добавлена экспериментальная строка и создан отдельный коммит.

Листинг 5 - отмена коммита с сохранением истории

```
git add README.md
git commit -m "Add experimental note"
git revert --no-edit HEAD
```

Команда revert создала новый коммит с обратным изменением. Предыдущий коммит остался в истории, а экспериментальная строка была удалена.

Обмен изменениями проверен через локальный репозиторий origin.git и вторую рабочую копию review. Репозиторий без рабочей папки создан в общей папке стенда. Команды remote add и push выполнены из первой рабочей копии.

Листинг 6 - создание общего репозитория и отправка коммитов

```
git init --bare --initial-branch=main origin.git

# В первой рабочей копии
git remote add origin ../origin.git
git push -u origin main

# В общей папке стенда
git clone origin.git review
```

Во второй копии создан файл review.txt с результатом проверки. После его коммита и отправки первая копия получила обновления командами fetch и pull. В этом опыте обмен происходил локально, без публикации учебного стенда на GitHub.

Листинг 7 - изменения во второй копии и обновление первой

```
# Во второй рабочей копии review
git add review.txt
git commit -m "Add review result"
git push

# В первой рабочей копии
git fetch origin
git pull --ff-only
git status
```

После обновления в первой копии появился review.txt. Команда status подтвердила чистое состояние рабочей папки. Параметр --ff-only разрешает обновление ветки только без создания дополнительного коммита слияния.

Листинг 8 - проверка истории и отметка результата

```
git log --graph --oneline --all --decorate
git tag v1.0
git status
```

История отражает первоначальный коммит, изменения в двух ветках, разрешение конфликта, экспериментальную правку, её отмену и получение коммита из второй копии. Тег v1.0 отмечает итоговое состояние работы.

Вывод Git при заключительной проверке:

```
nothing to commit, working tree clean
```

Терминал позволяет явно управлять индексом, ветками и обменом коммитами. Его достоинства - подробный вывод и возможность повторить последовательность действий. Недостатки - необходимость помнить команды и контролировать текущую папку и ветку.

В ходе работы выполнены создание репозитория, коммиты, ветвление, разрешение конфликта, отмена локального изменения и коммита, отправка и получение обновлений. Команды, пояснения и снимки экрана приведены непосредственно в отчёте.

Источники

Chacon S., Straub B. Pro Git. Разделы о сохранении изменений, ветвлении и слиянии. <https://git-scm.com/book/ru/v2>. Дата обращения: 24.09.2026.